

London Companies Dashboard 2.0

Page · Tag

Project Overview

Name: London Companies Dashboard 2.0

Goal: Visualize London-based companies by geographic grid cell, filtered by industry (SIC codes).

Status: Phase 1 Complete (Base Architecture Validated). Ready for Phase 2 (Job Alerts Integration).

Core Achievement: Accurate distinct count of 16317 "Advertising a" companies with no over-counting.

2. Data Sources & Ingestion Pipeline

The project uses a Postgres database as the source of truth, fed by Python scripts.

Source Table	Purpose	Status in PBI
<code>companies</code>	Master list of UK companies (name, address, incorporation date).	Loaded
<code>company_sic</code>	Bridge table linking companies to their SIC codes.	Loaded
<code>dim_sic</code>	Lookup table mapping SIC codes to text descriptions.	Loaded
<code>companies_geo_clusters</code>	Aggregated geographic data (grid cells) for map bubbles.	Loaded
<code>companies_sic_flat</code>	Main Fact Table. Flattened view combining company details with geography for fast visualization.	Loaded
<code>job_alerts</code>	Raw job alert data from Indeed/LinkedIn.	Not Loaded (Phase 2)
<code>location_mappings</code>	Helper table for geocoding job alerts.	Not Loaded (Phase 2)

3. Data Model Architecture (Power BI)

Relationship Diagram Logic: The model relies on specific "Both Directions" filter flow to ensure slicers work correctly across the flattened fact table.

- `dim_sic` → `company_sic` (1:* , BOTH)
 - Purpose:* Allows the "Industry" slicer to filter the bridge table.
- `company_sic` → `companies_sic_flat` (1:* , BOTH)
 - Purpose:* Filters the main fact table based on the selected industry.
- `companies_geo_clusters` → `companies_sic_flat` (1:* , BOTH)
 - Purpose:* Links the aggregated map bubbles to the detailed company records.

Critical Constraint: The `BOTH` direction is required here because filters must travel from the dimension tables *down* into the fact table (`companies_sic_flat`) to drive the visuals.

4. Key Measures & Logic

A. Exclusion Logic (The "Dormant" Filter) To prevent dormant companies from inflating counts, a calculated column `IsExcluded` was created in `companies_sic_flat`.

- **Helper Table:** `ExcludedCompanies` (Calculated Table)

DAX ▾

```
ExcludedCompanies =  
CALCULATETABLE(  
    DISTINCT('companies_sic'[company_number]),  
    'companies_sic'[sic_text] IN {"Dormant Company", "Non-trading  
company"}  
)
```

- **Column:** `IsExcluded` (Calculated Column in `companies_sic_flat`)
 - *Logic:* Checks if the current row's `company_number` exists in the `ExcludedCompanies` table. Returns TRUE/FALSE.
- **Result:** The main `Company Count` measure filters these out.

B. Primary Metrics

- **Company Count:** `DISTINCTCOUNT('companies_sic_flat'[company_number])` where `IsExcluded` is FALSE.
- **Bubble Count:** Used specifically for map sizing; respects the same distinct logic.

5. Known Constraints & Quirks

1. **SIC Crosswalks:** Some SIC text values map to multiple codes. The use of `DISTINCTCOUNT` prevents double-counting these duplicates.
2. **Grid Resolution:** Geographic clusters are frozen at ~11m resolution (`ROUND(lat,4)`). Changing this would break existing aggregates.
3. **Missing Geodata:** ~78k grid cells exist, but companies with NULL lat/lon are excluded from clustering.

6. Phase 2 Roadmap (Next Steps)

1. Load `job_alerts`: Import the Postgres table containing matched job postings.
2. **Establish Relationship:** Link `job_alerts[matched_company_number]` to `companies_sic_flat[company_number]`.

- *Note:* You will likely need to set this relationship to **Single Direction** (Filter flows from Companies to Jobs) initially to avoid ambiguity, unless you find filters aren't working.

3. **New Measures:** Create `Job Alert Count` using `DISTINCTCOUNT(job_alerts[alert_id])`.